

Redirecting postgres-mcp

The cloud gateway on clubuntu can now reach into this laptop's own Postgres, per call, given a host and credentials — the first of the two MCP servers meant to be pointable at any machine.

```
tool: run_query, connection_uri override    2026-08-16
```

00 The objective, restated

`docker-mcp` and `postgres-mcp` were both built with a per-call target override from the start — `docker_host` for the former, `connection_uri` for the latter — so one centrally-hosted gateway could reach any Docker daemon or Postgres database, not just the one it happens to run next to. `postgres-mcp` goes first here because it's the simpler of the two: a database connection string versus an SSH-authenticated remote Docker socket.

01 What was already true

Checking the source before changing anything: every one of `postgres-mcp`'s four tools — `list_schemas`, `list_tables`, `describe_table`, `run_query` — already accepts an optional `connection_uri`. When it's given, `tools/_scope.py`'s `scoped_query()` opens a single throwaway `asyncpg` connection to that exact DSN and closes it when the call ends, leaving the server's own default connection pool untouched. The application code needed *zero* changes — this had already been built.

02 The actual gap: nothing to connect to

`docker-compose.yml`'s `postgres` service had no `ports:` entry at all — not published to the host, let alone the tailnet. A `connection_uri` pointed at this laptop from clubuntu would have had nothing to reach.

BEFORE

postgres container
reachable only inside codercommands-net

AFTER

100.127.101.48:5433 → :5432
bound to the Tailscale IP only

not 0.0.0.0 — that would expose it to the whole LAN,
not just clubuntu over the tailnet

The default in `.env.example` stays `127.0.0.1` — a clone of this repo is closed by default until someone deliberately sets their own Tailscale IP.

03 Implementation

FILE	CHANGE
<code>docker-compose.yml</code>	<code>postgres</code> now publishes <code>\${POSTGRES_TAILNET_BIND:-127.0.0.1}:\${POSTGRES_HOST_PORT:-5433}:5432</code>
<code>.env</code> (local, not tracked)	<code>POSTGRES_TAILNET_BIND=100.127.101.48</code> , <code>POSTGRES_HOST_PORT=5433</code>
<code>.env.example</code>	documents both variables, defaulting to loopback so a fresh clone stays closed

Port `5433`, not `5432` — this laptop already runs an unrelated Postgres container from the separate `../POSTGRES` project on the default port.

04 An honest detour: the stale tool cache

Testing this the obvious way — calling the registered `postgres` MCP tool directly — produced a tool list that didn't match the source at all: zero-parameter `list_schemas`, plus tool names like `execute_sql` and `analyze_db_health` that belong to the third-party `crystaldba/postgres-mcp` package, not this project's own four tools. That was worth stopping for rather than trusting.

Checked directly on clubuntu: `pip show postgres-mcp` inside the running container confirmed `Version: 0.1.0`, *"MCP server for ad-hoc Postgres access, with a per-call connection_uri override"* — this project's own package, correctly deployed. The mismatch was local: this session's cached MCP tool list for `postgres` was left over from before the mid-session registration swap and had never refreshed. Verification continued with direct

MCP protocol calls over HTTP instead of the stale cached tool — a fresh Claude Code session picks up the real one.

05 Proof: two different databases

The same query, run twice through the same cloud `postgres-mcp` — once with `connection_uri` set, once without:

```
SELECT INET_SERVER_ADDR()::TEXT, (SELECT COUNT(*) FROM SYNC_RUNS)

run_query(connection_uri="postgresql:// ... @100.127.101.48:5433/codercommands")
  → server_ip: 172.28.0.2    sync_runs: 10

run_query()    -- no override, clubuntu's own default
  → server_ip: 192.168.96.3  sync_runs: 5
```

Different internal Docker network addresses, different row counts — genuinely two separate databases on two separate machines, both reached through the exact same tool call, distinguished only by one parameter.

CHECK	RESULT
Raw TCP reachability, clubuntu → laptop:5433	✓ connected over Tailscale
<code>list_schemas</code> via override	✓ returned this laptop's real schemas
<code>run_query</code> via override vs. default	✓ two distinct server IPs, two distinct row counts
Port closed to LAN/internet, tailnet-only	✓ bound to <code>100.127.101.48</code> , not <code>0.0.0.0</code>
Application code changes required	<i>none — already built</i>

06 Live usage, in practice

Beyond the internal-IP comparison in §05, the same override was exercised the ordinary way — the exact shape a real caller uses. Every tool takes `connection_uri` as one more argument alongside whatever else it needs; nothing else about the call changes.

```
CONNECTION_URI FORMAT

postgresql:// <user>:<password>@<host>:<port>/<database>
```

```
# pointed at this laptop specifically:  
postgresql://codercommands:<password>@100.127.101.48:5433/codercommands
```

`list_tables`, called with that URI, returned this laptop's real schema — not clubuntu's:

```
list_tables(connection_uri=...) → commands, sync_runs
```

`run_query`, same URI, pulled this laptop's actual sync history — matching exactly what the local `sync` container has been doing:

```
SELECT ID, STATUS, STARTED_AT FROM SYNC_RUNS ORDER BY ID DESC LIMIT 5
```

```
id=10  success  2026-08-16T16:17:59Z  
id=9   success  2026-08-15T21:52:48Z  
id=8   success  2026-08-15T21:35:51Z  
id=7   success  2026-08-15T20:36:49Z  
id=6   success  2026-08-15T14:36:27Z
```

Both calls went through the same path as §05: this laptop's Claude Code → cloud gateway on clubuntu → `postgres-mcp` → back out over Tailscale to `100.127.101.48:5433`. The only thing distinguishing "reach clubuntu's own database" from "reach this laptop's" was the presence of one argument.

07 What's next

`docker-mcp` is the harder half of this objective — its override is an SSH-authenticated `docker_host=ssh://...` URL, and the container currently has no SSH client or private key installed to make that connection with. That's deliberately deferred to its own doc.

```
final-run - 4 · postgres-mcp redirect proven end to end over Tailscale
```